



QuizMaster: An Adaptive Formative Assessment System

Fuhua Lin¹ , Raymond Morland¹  , and Hongxin Yan²

¹ Athabasca University, Athabasca, AB T9S 3A3, Canada
oscarl@athabascau.ca, rmorland1@learn.athabascau.ca

² School of Computing, University of Eastern Finland, Joensuu, Finland
hongya@student.uef.fi

Abstract. In this paper, we introduce QuizMaster, an innovative web-based adaptive learning system designed for conducting formative assessment on-demand anytime during students' course study. QuizMaster reduces learner time spent on assessment and accelerates formative feedback delivery. Leveraging a Multi-Armed Bandit algorithm for question sequencing and feedback, it ensures intelligent assessment processes. Additionally, we employ Large Language Models to auto-generate questions, enhancing instructor productivity. When deployed, QuizMaster will serve to assess adaptive algorithms for formative assessment in real-world learning scenarios. Through our detailed analysis of the QuizMaster architecture, we demonstrate how to leverage reinforcement learning and generative intelligence in the development of systems for formative assessment.

Keywords: Adaptive Learning Systems · Intelligent Tutoring Systems · Multi-armed Bandits · Reinforcement Learning · Personalized Learning · Formative Assessment · Generative Intelligence

1 Introduction

Formative assessment can be an integral part of the learning process in formal educational settings [1]. It differs from summative assessment in that it is meant to be an assessment *for* learning and not an assessment *of* learning through grading [2]. The goal of formative assessment is to provide individualized feedback to enhance learners' understanding of the subject materials and guide their studies. Therefore, formative assessment aids the student by helping them bridge the gap between their knowledge state and their learning goal, i.e., mastery of the learning objectives (LOs). While formative assessment can be fundamental to a student's learning process, current self-quizzes for formative assessment face two significant limitations. First, feedback provided in the self-quizzes may be generic and lack the depth needed for students to gain a nuanced understanding of their mistakes, hindering targeted improvement. Second, developing high-quality self-quizzes requires extensive resources, time, and expertise. Institutions with limited resources may face challenges creating effective, diverse, and engaging quiz content. Thus, we aim to provide a solution to both problems by employing course modelling, reinforcement learning techniques, and large language models in a web-based adaptive formative assessment system – QuizMaster, introduced in this paper.

QuizMaster is a fully-fledged client-server application comprised of web-based interfaces, an application server, and a database. The goal of QuizMaster is to shorten the assessment-feedback cycle of formative assessment. It accomplishes this goal in three ways: a) reducing the time that learners spend on assessments, b) providing instantaneous automated feedback to the learner, and c) decreasing the time instructors spend developing content for assessments. In these ways, QuizMaster can increase efficiency for both students and instructors. To increase the learning efficiency for students, QuizMaster employs a Multi-armed Bandit algorithm to provide adaptive feedback and question sequencing. To increase the question creation efficiency for instructors, QuizMaster leverages large language models to create questions for assessments. These approaches together allow QuizMaster to provide a learning environment for students to undergo efficient formative assessment and receive individualized feedback.

The remainder of this paper is organized as follows. Related work and background information are reviewed in Sect. 2. The QuizMaster implementation is discussed in Sect. 3. In Sect. 4, evaluations of the various components and our proposed testing, both simulation and in-person, are discussed. Finally, in Sect. 5, we conclude the paper with a discussion about the system and future work.

2 Background and Related Work

Reviews of systems designed to provide automated feedback have been performed previously [3], including for specific domains such as programming instruction [4]. McLaughlin and Yan (2017) reviewed various delivery methods for formative assessment [5]. While the research into tools for formative assessment is extensive, most of these systems are limited in scope and are specialized in their domain. We seek to develop a generalized system that can be used for different knowledge domains. To this end, we need a simple and general algorithm to provide adaptive formative feedback.

Traditional adaptive learning systems (ALS) focus on forming a robust student model to estimate a learner’s knowledge state. This estimated knowledge state, or mastery of knowledge components, is used for question sequencing. Traditional ALS algorithms include Bayesian and Logistic Knowledge Tracing [6], Item Response Theory [7], and Partial-Observable Markov decision processes [8]. While research into these algorithms is extensive, they do have limitations. These algorithms typically require a set of parameters that are difficult to tune [9, 10]. They may also require making assumptions about a learner’s mastery of a subject beforehand. This may require pretesting or relying on self-assessment by the learner, potentially leading to inaccurate initial data being used to calibrate parameters. To address these limitations, we have chosen to utilize Multi-armed Bandit (MAB) algorithms.

MABs are a class of reinforcement learning algorithm that seeks to provide a solution to the exploitation-exploration problem. MABs are a recent addition to the field of adaptive learning. A recent survey by Mui et al. (2021) explores the various use cases of MABs in ALSs [11]. MAB algorithms have been shown to be simpler to implement while remaining as effective as traditional algorithms when used for formative assessment [9, 13]. Lattimore and Szepesvári (2020) provide a comprehensive introduction to the topic of MABs [12]. The goal of an original MAB is to maximize the total reward they receive

while minimizing the regret of making choices. The decisions surrounding managing this trade-off and the choice of sampling policies they use differentiate MAB algorithms. By using an MAB algorithm, we show how the number of questions asked during assessment to detect mastery can be reduced, thus increasing student efficiency.

One of the most challenging and time-consuming aspects of formative assessment is the generation of appropriate questions. The questions asked in an assessment must be relevant to the LOs of the course. Therefore, subject matter experts with sufficient domain knowledge usually create the questions for the course. However, due to the substantial cost of time and resources that question generation requires, developing systems that automatically generate questions is an active area of research [14]. We aim to provide efficient ways for instructors to generate effective questions for use with QuizMaster by turning to large language models (LLMs) for assistance.

With the recent developments in LLMs, there has been a growing interest in their use for building and integrating with ALSs. One application of LLMs in ALS that may prove fruitful is in the generation of questions for assessments [15]. The use of generative AI systems such as OpenAI's ChatGPT may lead to an increase in instructor productivity by reducing the time they spend on manually crafting course content. One example of this increase in productivity is using these models to generate programming questions by integrating an LLM with a web-based user interface [16]. However, LLMs are not yet in a state of development where it is safe to use them without humans in the loop. Therefore, while an increase in efficiency may be realized by instructors when creating content, an expert's evaluation of the questions is still required before they are presented to students [14].

3 System Overview

One of the primary goals of QuizMaster is to contribute to the research and development of functioning formative assessment systems. Therefore, we provide the following thorough explanation of the system architecture and each of the various modules that QuizMaster is composed of.

3.1 System Architecture

QuizMaster is implemented using a client-server architecture (see Fig. 1). This architecture allows for flexibility in separating the concerns of the client applications and the server application. Two client applications accessing a single server application, one for students and another for instructors, have been designed and developed in the architecture. Both the student and instructor applications utilize the open-source ReactJS framework¹.

Student Interface. The student interface consists of an assessment component and a student profile. Through the assessment component, students can take quizzes and receive formative feedback. It provides the presentation for the quizzes while the server executes the logic of the adaptive algorithms. From the assessments home screen, the

¹ <https://react.dev/>.

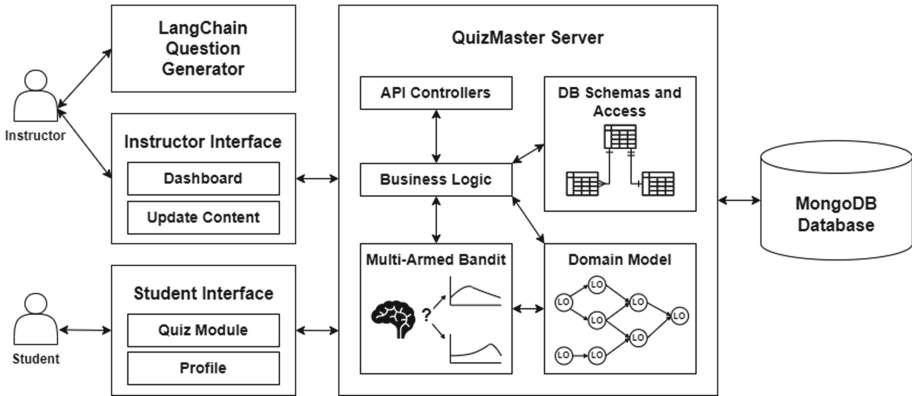


Fig. 1. The System Architecture of QuizMaster

student selects a course and a unit from that course to begin a quiz. The student can only choose from courses for which they are registered. After they have chosen the course, they will be able to select the unit they want to be tested in. Presently QuizMaster supports multiple-choice and interactive questions. The interactive data structures and algorithm questions are sourced from OpenDSA² (see Figs. 2 and 3). Future work will involve integrating other assessment types.

During a quiz the student is presented with a single question at a time. They select their answers and press the submit button. While a timer indicating the amount of time a user has been on a question is shown below the questions, a timer indicating the total time spent on the quiz appears in the upper right-hand corner. The number of questions that a student will be presented with is determined by the MAB algorithm. However, the number of questions will not be more than the budget of the quiz. The budget is displayed beneath the question beside the timer. After the student completes a quiz, they are shown the results view (see Fig. 7). This page provides summative feedback as well as the formative feedback including the weak LO and the recommended remedial learning materials. Along with the formative feedback, the students' chain of weak learning objectives is displayed as a graph. This serves to recommend the sequence of LOs that the student should study to improve their proficiency in their weak LO. How this chain of weak LOs is generated is discussed in Sect. 3.5. The student interface also has a profile page where students can view the results of their completed quiz sessions and update their profile information.

Instructor Interface. The instructor interface allows instructors to create and maintain course models. When an instructor signs in, they are presented with the courses assigned to them. Instructors can view and update the information associated with each course, including the individual units, learning objectives, quiz parameters, and questions. There are two types of multiple-choice questions present in QuizMaster: expert-generated and LLM-generated. The LLM-generated questions will be further discussed in the Content Library section. The expert-generated questions are developed by the instructor and then

² <https://opensa-server.cs.vt.edu/>.

The screenshot shows the QuizMaster interface. At the top, the header "QuizMaster" is on the left and a timer "01 : 00" is on the right. The main content area displays a question: "Which of the following is not a programming language?". Below the question are three radio button options: "Java", "C++", and "Map". At the bottom of the question box, it says "Learning Objective 1" and "00 : 53 3 / 20". Below the question box is a "Select Answer" button. At the bottom of the screen, a dark blue footer contains the text "COMP 272: Data Structures and Algorithms Unit 1: Foundations of Computer Science" on the left and a red "End Quiz" button on the right.

Fig. 2. Example of a multiple-choice question. The student selects what they think the best option is and submits it. Another question is presented if a stopping criterion is not met.

The screenshot shows the QuizMaster interface for an interactive question. The header "QuizMaster" is on the left and a timer "02 : 13" is on the right. The main content area displays a question about the DFS algorithm. It includes instructions: "Reproduce the behavior of the DFS algorithm for the graph below. Just click on the edges in the order that they will be traversed by the DFS algorithm. Start with Node A. If there is more than one node that could be visited next, choose the one that comes first in alphabetical order." Below the instructions is a graph with nodes A, B, C, D, E, G, and F. Node A is highlighted in yellow. The graph shows edges between A and B, A and C, B and G, B and E, C and E, D and E, D and G, and F and B. At the bottom of the question box, it says "Score: 1 / 6, Points remaining: 3, Points lost: 2". Below the question box is a "Select Answer" button. At the bottom of the screen, a dark blue footer contains the text "COMP 272: Data Structures and Algorithms Unit 1: Foundations of Computer Science" on the left and a red "End Quiz" button on the right.

Fig. 3. Example of an interactive-type question. This figure shows that a student is performing an auto assessed graph question. We can observe that the student has scored one out of six points, there are three points remaining, and the student has lost two points. We use the open-source material provided by OpenDSA for these interactive questions.

Dashboard > Content > Questions > New

New Multiple-Choice Question

CancelSave

1Info2Content3Preview

Step 2 (Optional): Generate Content with LLM

LLM Model

LLM Model

Auto-Generate Content

Step 3: Add content to the body of the question

Question Content

Normal B I U S " " /> f_x x_1 x^2 A [] = < /> I_x

What does the following code do?

```
function hello() {
  console.log("Hello world!");
}
```

Formula Help: [KATEX Documentation](#)

Step 4: Add Options to the Question

Option Editor

B I U S " " /> f_x x_1 x^2 I_x

Feedback

☐ Option is Correct

Save Option Clear Editor

Options	Is Correct	Edit Option	Delete Option
1.Prints "Hello World!" to the console.	Yes		
2.Adds two numbers.	No		

PreviousNext

Fig. 4. The Instructor interface for creating multiple-choice questions. This screenshot shows the question content step in the question generation form. At this step the instructor adds the content to the body of the question as well as the options.

input into the system manually (see Fig. 4). The LLM-generated questions produced using separate scripts are entered manually into the form after the instructor validates them.

Rich text format is used for question creation, allowing different formatting and variability in content so that the questions can be more than only plain text. For instance, code can be displayed using code blocks with formatting that depends on the programming language. Images can also be included in the content of the question.

When entering questions, the instructor must select the appropriate learning objective to associate with them. This association will automatically include them as potential candidate questions when the MAB algorithm selects their LO during a quiz. This mechanism is another time-saving benefit QuizMaster offers, along with having the LLM generate questions. Here, instructors only need to enter the questions without manually sequencing the questions for each assessment. This method can save instructors precious time that can be spent on more important teaching tasks.

Server Application. The server, implemented as a NodeJS³ application, consists of modules for the API controller, business logic, MAB, domain model, and database access and schemas, as shown in Fig. 1. The modularity of the server architecture gives the application flexibility when components need to be added, updated, and removed. This flexibility will be especially important for QuizMaster to be developed into a robust and versatile ALS platform.

Database. All the application data entities and content are stored as documents in a MongoDB⁴ database. The NoSQL document-based MongoDB gives us the flexibility and ease of use we need for QuizMaster.

3.2 Content Library

The QuizMaster content library consists of the questions available for quizzes. QuizMaster currently focuses on multiple-choice and interactive-type questions. These questions can be auto-graded in formative assessment for immediate feedback because they have definite answers. The interactive questions, obtained from the open-source materials provided by OpenDSA, integrate directly into the student application. We chose to integrate OpenDSA questions because initial testing will focus on the content from a course in data structures and algorithms. Including interactive exercises will allow us to evaluate the effectiveness of having incorporated different question types.

To improve instructor efficiency, LLM-generated multiple-choice questions are included in the content library after experts verify their validity. The LLM-generated questions QuizMaster uses are generated with LangChain⁵ using OpenAI's GPT3.5 and GPT4 models as described by Jhaji et al. in [15]. Instructors must manually verify each question before they submit them to the database. They can edit them to their liking from within the instructor interface. An example of a question generated using the LangChain question generator module is shown in Fig. 5.

³ <https://nodejs.org/>.

⁴ <https://www.mongodb.com/>.

⁵ <https://www.langchain.com/>.

Sure! Here are ten multiple choice questions on the concept of data structures, each with four options and an explanation for the correct answer.

Question 1: Which of the following is a type of data structure?

A) Java
B) Binary Tree
C) HTML
D) JavaScript

Correct Answer: B) Binary Tree

Explanation: A binary tree is a data structure in which each node has at most two children, which are referred to as the left child and the right child.

...

Fig. 5. An example response from the LangChain question generator when prompted to return ten multiple choice questions on the concept of data structures.

3.3 Course Model

One essential component of QuizMaster is the course model. The course model specifies the relationships among the set of knowledge components in a course. Students must progress through the course model by exposure to each of the knowledge components in an appropriate way for learning. In QuizMaster, the course model represents the prerequisite relationships among LOs of a section and/or unit in a course. These relationships are represented as an AND/OR graph described in [17]. The graph structure of the course model is considered when updates are made to the student model for LOs and when the system searches for weak LOs. Figure 6 illustrates an example of course models, the course model for COMP272 Data Structure used in QuizMaster.

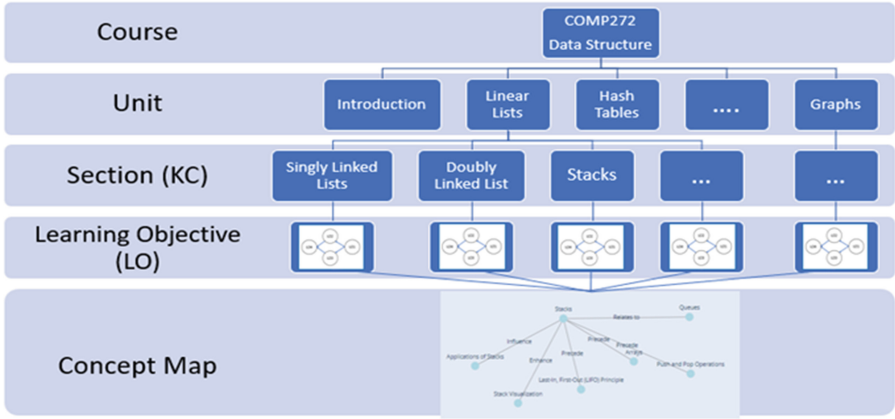


Fig. 6. An example of course models used in QuizMaster for a data structures and algorithms course.

3.4 Question Sequencing and Student Model

QuizMaster utilizes an MAB algorithm for question sequencing and mastery detection. The MAB technique that QuizMaster utilizes is the hybrid algorithm for the Dilemma of Confidence HDoC algorithm described by Kano et al. [18], which is modified to find the weak arms as described in [19]. The HDoC algorithm is a form of upper confidence bound (UCB) sampling policy that estimates whether an arm is strong within a degree of confidence above a threshold. In QuizMaster, the arms of the bandit correspond to the LOs, so pulling an arm means asking the student a question. A student's answers to a question are binary, either right or wrong. Thus, the student's proficiency for a given LO can be modelled as a Beta distribution with the alpha and beta parameters corresponding to the correct and incorrect answers, respectively. This model of student progress defines the student model of the system. It represents an estimate of the student's knowledge state during the quiz. The task of the MAB algorithm is to determine the next LO to draw a question from so that sufficient evidence can be collected to determine whether the LO is a strong or weak one for the student. The QuizMaster MAB seeks to find weak LOs for a student to invest more learning efforts in.

The QuizMaster question sequencing and mastery detection work as follows. Initially, the system has no information on the student's knowledge state for the LOs in a unit; therefore, a pre-sample is conducted. In the pre-sample, each LO is sampled an equal number of times, with questions drawn randomly from the LOs. The number of pre-sampling rounds is a parameter set by the instructor. Responses, in the form of answers, are sent to the server, and the parameters for the beta distributions of each sampled LO are updated accordingly. This update may have cascading effects, known as pseudo-rewards, as the LOs with which the sampled LO has prerequisite relationships may also be updated. Once the pre-sample is completed, the main loop of the MAB algorithm is entered. After the LO update is complete with the final pre-sample question being answered, the stopping criterion is checked. The quiz will stop when one of the following conditions is met:

- The budget is reached, i.e., the maximum number of questions allowed is asked.
- The algorithm determines that there are no weak LOs, i.e., all LOs are strong.
- The algorithm is confident that it has found one or more weak LO.

If no stopping criterion applies, the MAB samples another LO. The MAB determines which LO to draw a question from based on the HDoC sampling policy. The system presents the student with an available question from that LO, chosen at random. The system will continue to ask questions in this sample-update-check loop until one of the stopping criteria is met.

3.5 Formative Feedback Generation and Presentation

At the end of a quiz session when a stopping criterion is met, the results of the quiz and the formative feedback are presented to the student (see Fig. 7). To produce the formative feedback, the chain of weakness in the LO-graph is calculated. A graph search is conducted to calculate the foremost weak LO, as described by Lin and De Silva in [17]. This search finds the weak LOs that have the most significant learning potential.



Fig. 7. Assessment results screen shown to the student after they have completed an assessment. The screen shows the formative feedback provided to the student and a recommended learning path. Students can access this screen from their profile by clicking on the corresponding completed quiz session.

That is, they are the LOs that may be the optimal ones for the student to spend time improving upon. This is an active line of research in our group, hoping to generate even more personalized feedback corresponding to individuals' weaknesses.

4 Evaluation of Methods and Future Testing

QuizMaster is the culmination of several research studies into algorithms and techniques for formative assessment in adaptive learning within the Intelligent Educational Systems research group at Athabasca University, Canada. Preliminary simulation tests have been conducted on the HDoC sampling policy used in QuizMaster's MAB algorithm [19]. Using AI students, these simulations have shown promising results for reducing the number of questions asked before a weak LO could be found. Evaluations of the LLM question generation technique have shown it is effective at creating domain-specific multiple-choice questions [15]. We will use the questions generated for this study in our student and simulation testing. Other work in our group has also shown how LLMs can be used in adaptive practicing systems [20]. The technique for finding the chain of weak LOs within the graph-based domain model is based on the results of simulation testing to generate adaptive feedback for formative assessment [17]. The simulations in this study showed that MABs combined with a graph-based domain model can be effective at finding weak LOs in formative assessment. Now that QuizMaster has integrated these techniques, they can be evaluated with real students.

Testing with real students will focus on assessing a set of LOs for an intermediate-level data structures and algorithms course. We will quiz students with a version of expert-generated questions and a version of LLM-generated questions to compare the effectiveness of different question-generating techniques. We will also compare the HDoC and other MAB sampling algorithms by evaluating the student's mastery and question sequencing effectiveness. Our testing will also include an opportunity for students and instructors to provide us with feedback about using the system so that we can further enhance the user experience and interaction.

5 Conclusions and Future Work

We introduced QuizMaster, a fully-fledged web-based formative assessment system. It can save students time on assessment, providing immediate feedback to learners on weak learning objectives, and increasing instructor efficiency in creating quiz questions. Through our discussion of the QuizMaster architecture, we demonstrate its effectiveness for providing formative assessment by integrating reinforcement learning and generative intelligence. This makes QuizMaster a valuable addition to the general formative feedback applications discussed in the literature. It is important to present fully engineered systems in academic literature to benefit others who need them.

As discussed above, the next step for developing the QuizMaster system will be testing with students in real-world learning scenarios. After this testing, we will continue to develop the QuizMaster system so that it can manage more general learning use-cases. Future enhancements to QuizMaster will include the auto-generation of questions in the instructor interface as well as more personalized feedback generation. The potential for

LLM and other generative technologies to enhance systems such as ours is significant, and we will continue to research ways to leverage these technologies.

Finally, the end goal of our testing and development is for QuizMaster to become a fully general formative assessment tool. We intend for QuizMaster to be capable of integrating with popular learning management systems using the learning tools interoperability (LTI) standard. This will allow QuizMaster to be used alongside a vast number of applications, helping both students and instructors reach their goals more efficiently.

Acknowledgments. We acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC), Alberta Innovates, and Athabasca University, Canada. Additionally, we thank Michael Dupuis, Supun De Silva, and Diptanshu Mandal for their efforts in implementing the prototype of QuizMaster.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Black, P., Wiliam, D.: Assessment and classroom learning. *Assess. Educ. Princ. Policy Pract.* **5**(1), 7–74 (1998). <https://doi.org/10.1080/0969595980050102>
2. Bennett, R.E.: Formative assessment: a critical review. *Assess. Educ. Princ. Policy Pract.* **18**(1), 5–25 (2011). <https://doi.org/10.1080/0969594X.2010.513678>
3. Deeva, G., Bogdanova, D., Serral, E., et al.: A review of automated feedback systems for learners: classification framework, challenges and opportunities. *Comput. Educ.* **162**, 104094 (2021). <https://doi.org/10.1016/j.compedu.2020.104094>
4. Keuning, H., Jeuring, J., Heeren, B.: A systematic literature review of automated feedback generation for programming exercises. *ACM Trans. Comput. Educ.* **19**(1), 3:1–3:43 (2018). <https://doi.org/10.1145/3231711>
5. McLaughlin, T., Yan, Z.: Diverse delivery methods and strong psychological benefits: a review of online formative assessment. *J. Comput. Assist. Learn.* **33**, 562–574 (2017). <https://doi.org/10.1111/jcal.12200>
6. Pelánek, R.: Bayesian knowledge tracing, logistic models, and beyond: an overview of learner modeling techniques. *User Model. User-Adap. Inter.* **27**, 313–350 (2017). <https://doi.org/10.1007/s11257-017-9193-2>
7. Jia, J., Le, H.: The design and implementation of a computerized adaptive testing system for school mathematics based on item response theory. In: Lee, L.K., Hou U, L., Wang, F.L., Cheung, S.K.S., Au, O., Li, K.C. (eds.) *ICTE 2020. CCIS*, vol. 1302, pp. 100–111. Springer, Singapore (2020). https://doi.org/10.1007/978-981-33-4594-2_9
8. Rafferty, A.N., Brunskill, E., Griffiths, T.L., Shafro, P.: Faster teaching by POMDP planning. In: Biswas, G., Bull, S., Kay, J., Mitrovic, A. (eds.) *AIED 2011. LNCS*, vol. 6738, pp. 280–287. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-21869-9_37
9. Melesko, J., Ramanauskaitė, S.: Time saving students’ formative assessment: algorithm to balance number of tasks and result reliability. *Appl. Sci.* **11**, 6048 (2021). <https://doi.org/10.3390/app11136048>
10. Clement, B., Roy, D., Oudeyer, P.-Y., Lopes, M.: Multi-armed bandits for intelligent tutoring systems. *J. Educ. Data Min.* **7**(2), 20–48 (2015). <https://doi.org/10.5281/zenodo.3554667>

11. Mui, J., Lin, F., Dewan, M.A.A.: Multi-armed bandit algorithms for adaptive learning: a survey. In: Roll, I., McNamara, D., Sosnovsky, S., Luckin, R., Dimitrova, V. (eds.) AIED 2021. LNCS, vol. 12749, pp. 273–278. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-78270-2_49
12. Lattimore, T., Szepesvári, C.: Bandit Algorithms. Cambridge University Press, Cambridge (2020)
13. Melesko, J., Novickij, V.: Computer adaptive testing using upper-confidence bound algorithm for formative assessment. Appl. Sci. **9**, 4303 (2019). <https://doi.org/10.3390/app9204303>
14. Bachiri, Y. A., Mouncif, H.: Artificial intelligence system in aid of pedagogical engineering for knowledge assessment on MOOC platforms: Open EdX and Moodle. Int. J. Emerg. Technol. Learn. **18**(5), 144–160 (2023). <https://doi.org/10.3991/ijet.v18i05.36589>
15. Jhaji, G., Lin, F., Dewan, M.A.A., Yan, H.: LLM-powered question generation using domain models for formative assessment. Manuscript submitted for publication (2024)
16. Hassany, M., Brusilovsky, P., Ke, J., et al.: Authoring worked examples for Java programming with human-AI collaboration. arXiv preprint (2023). <https://doi.org/10.48550/arXiv.2312.02105>
17. Lin, F., De Silva, S.: An approach to generating adaptive feedback for online formative assessment. In: Frasson, C., Mylonas, P., Troussas, C. (eds.) ITS 2023. LNCS, vol. 13891, pp. 88–99. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-32883-1_8
18. Kano, H., Honda, J., Sakamaki, K., et al. Good arm identification via bandit feedback. Mach. Learn. **108**, 721–745 (2019). <https://doi.org/10.1007/s10994-019-05784-4>
19. Morland, R., Wang, L., Lin, F.: Fast Weakness identification for adaptive feedback. In: Lin, O., Sifaleras, A. (eds.) ITS 2024, LNCS, vol. 14798, pp. 40–54 (2024)
20. Kabir, M.R., Lin, F.: An LLM-powered adaptive practicing system. In: Proceedings of the Workshop on Empowering Education with LLMs - the Next-Gen Interface and Content Generation 2023, co-located with 24th International Conference on Artificial Intelligence in Education (AIED 2023), Tokyo, Japan (2023)